

OPERATING SYSTEM AND SYSTEM PROGRAMMING

CODE :25CS2104E

ROLL NO:2520030255

NAME :V SAI NIKHIL

WEEK-4 PRACTICAL

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid[3];
    int i;

    printf("Parent Process: PID = %d\n", getpid());

    // Create 3 child processes
    for (i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if (pid[i] == 0)
        {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            sleep(2);
            printf("Child %d completed\n", i + 1);
            exit(0);
        }
    }

    // Using wait()
    printf("\nParent is using wait()...\n");
    wait(NULL);
    printf("One child completed using wait()\n");

    // Using waitpid()
    printf("\nParent is using waitpid()...\n");

    for (i = 0; i < 2; i++)
    {
        waitpid(pid[i], NULL, 0);
        printf("Child %d completed using waitpid()\n", i + 1);
    }

    printf("\nAll child processes completed.\n");
    printf("Zombie processes eliminated using synchronization.\n");

    return 0;
}
```

```
vsainikhil@NIKHIL-BOOK:~$ vi program4.c
vsainikhil@NIKHIL-BOOK:~$ gcc program4.c -o program4
vsainikhil@NIKHIL-BOOK:~$ ./program4
Parent Process: PID = 843
Child 1: PID = 844
Child 2: PID = 845

Parent is using wait()...
Child 3: PID = 846
Child 1 completed
Child 2 completed
Child 3 completed
One child completed using wait()

Parent is using waitpid()...
Child 1 completed using waitpid()
Child 2 completed using waitpid()

All child processes completed.
Zombie processes eliminated using synchronization.
vsainikhil@NIKHIL-BOOK:~$ |
```